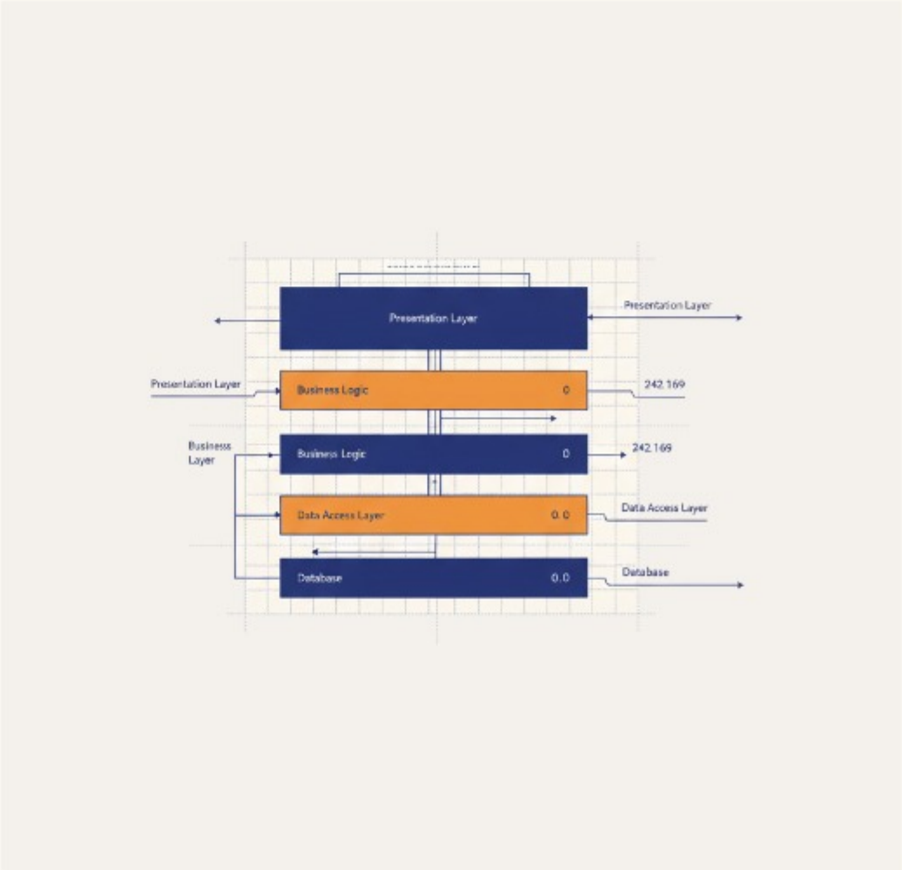
Arquitectura de Software

y Arquitectura en Capas

Fundamentos para Sistemas Escalables y Mantenibles

¿Qué es Arquitectura de Software?

- Definición: La estructura fundamental de un sistema de software
- Decisiones de alto nivel sobre cómo organizar el código
- Define componentes, relaciones y principios de diseño
- Es el "plano" o "blueprint" de tu aplicación



💡 Analogía: Como la arquitectura de edificios, antes de construir necesitas un plano

¿Por qué es importante?

Mantenimiento

Facilita el mantenimiento a largo plazo

Trabajo en Equipo

Permite que equipos trabajen en paralelo

Costos

Reduce costos de cambios futuros

Calidad

Mejora la calidad y confiabilidad del sistema

Onboarding

Facilita la incorporación de nuevos desarrolladores

⚠️ Sin arquitectura, el proyecto se vuelve caótico en 6-12 meses

Diseño vs Arquitectura

ARQUITECTURA

✓ Decisiones de **alto nivel**

✓ Estructura **general**

✓ **Difícil** de cambiar

> "¿Qué capas tendrá el sistema?"

DISEÑO

✓ Decisiones de **bajo nivel**

✓ Implementación **específica**

✓ Más **flexible**

> "¿Cómo implemento esta clase específica?"

Ambos son importantes y complementarios

💡 Arquitectura = ciudad; Diseño = casas individuales

Rol del Arquitecto de Software

1. Define la estructura y patrones del sistema
2. Toma decisiones técnicas críticas
3. Balancea requisitos funcionales y no funcionales
4. Comunica la visión técnica al equipo
5. Revisa y valida implementaciones

💡 No es solo "el desarrollador senior", requiere visión holística

EL PROBLEMA

Sistemas sin Arquitectura

- Todo el código en un solo archivo o carpeta
- Sin separación clara de responsabilidades
- Difícil entender qué hace cada parte
- Cambios pequeños rompen funcionalidades lejanas
- Imposible trabajar en equipo eficientemente

⚠️ Mostrar ejemplo de proyecto real desordenado si es posible

Código Espagueti ⚠️

- Código enredado sin estructura lógica
- Dependencias circulares entre componentes
- Lógica de negocio mezclada con UI y datos

Ejemplo:

```
Controller que hace consultas SQL directas y renderiza
HTML
```

Resultado: Mantenimiento costoso y bugs constantes

🍝 Llamado "espagueti" porque parece un plato de pasta enredada

Consecuencias Reales

Tiempo Multiplicado

El tiempo de desarrollo se multiplica con cada feature

Bugs Complejos

Bugs difíciles de rastrear y corregir

Escalar Imposible

Imposible escalar el equipo

Miedo a Cambiar

Miedo a modificar código existente

Deuda Técnica

Deuda técnica insostenible

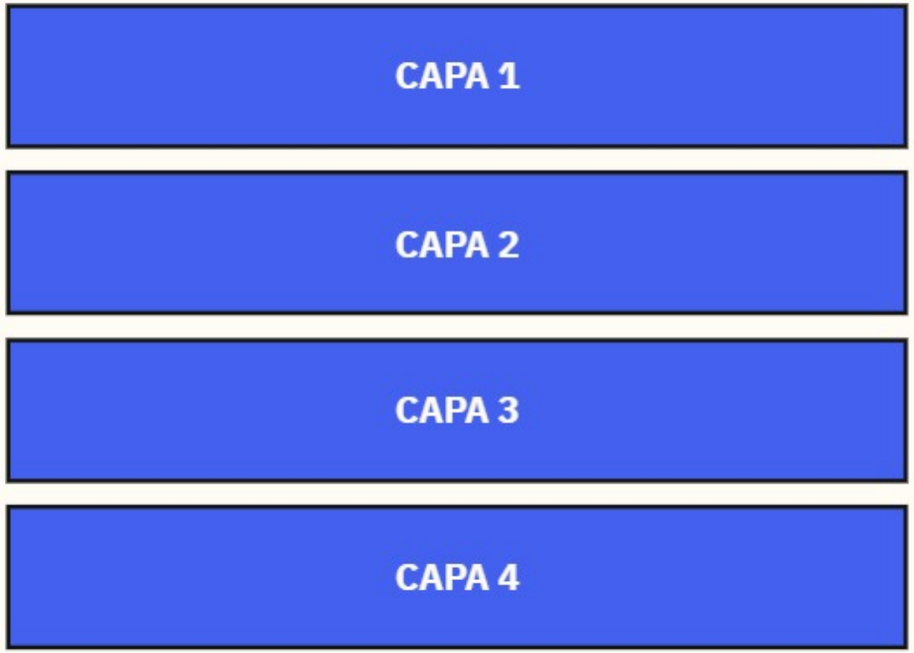
⚠️ Proyectos reales han sido reescritos completamente por esto

LA SOLUCIÓN

Arquitectura en Capas

- Patrón arquitectónico clásico y probado
- Organiza el código en capas horizontales
- Cada capa tiene responsabilidades específicas
- Las capas se comunican solo con capas adyacentes

También llamada "N-Tier Architecture"



💡 Es uno de los patrones más antiguos pero sigue siendo relevante

Objetivos de Arquitectura en Capas

Separación de responsabilidades
(Separation of Concerns)

Bajo acoplamiento entre componentes

Alta cohesión dentro de cada capa

Facilitar testing y mantenimiento

Permitir cambios sin afectar todo el sistema

💡 Estos principios aplican a muchas arquitecturas modernas

Principios Fundamentales

01 Conoce solo la capa inferior

Cada capa solo conoce la capa inmediatamente inferior

02 Flujo unidireccional

Dependencias fluyen en una sola dirección (hacia abajo)

03 Reemplazo independiente

Cada capa puede ser reemplazada independientemente

04 No saltar capas

No "saltar" capas en las llamadas

05 Interfaces bien definidas

Interfaces bien definidas entre capas

⚠️ Violaciones comunes: UI llamando directamente a la base de datos

Las Capas Típicas

Capa de Presentación

(UI / Controllers)

Capa de Negocio

(Business Logic)

Capa de Repositorios

(Repositories)

Capa de Datos / Infraestructura

(Entities / Database / External Services / Frameworks)

↓
↓
↓

💡 Algunos modelos combinan capas, no hay un estándar único

Capa de Presentación

RESPONSABILIDAD:

Interacción con el usuario

CONTIENE:

Controllers, Views, DTOs, Validators de entrada

NO DEBE CONTENER:

Lógica de negocio, acceso a datos directo

EJEMPLO:

`UserController que recibe HTTP requests`

TECNOLOGÍAS:

React, Angular, ASP.NET MVC, Express routes

💡 Solo traduce entre el mundo exterior y tu aplicación

Capa de Negocio

RESPONSABILIDAD:

Reglas de negocio y lógica central

CONTIENE:

DTOs, Servicios, Contratos

NO DEBE CONTENER:

Dependencias de frameworks, UI, base de datos

EJEMPLO:

`Clase User con método CanPurchase() que valida reglas de negocio`

PRINCIPIO:

Es el corazón del sistema, independiente de tecnología

💗 Esta capa debe poder probarse sin base de datos ni UI

Capa de Repositorios

RESPONSABILIDAD:

Capa intermediaria entre la capa de negocio y la capa de datos, encargada de contener las operaciones CRUD.

CONTIENE:

Repositorios base (Genérico), Repositorios personalizados

NO DEBE CONTENER:

Lógica de negocio

EJEMPLO:

`IClienteRepository con metodo AddAsync()`

RESPONSABILIDAD:

Mantener desacopladas las operaciones de base de datos del negócio.

🔧 Todo lo que puede cambiar por decisiones técnicas va aquí

Capa de Datos

RESPONSABILIDAD:

Persistencia y acceso a datos

CONTIENE:

DbContext, Dependencias externas

NO DEBE CONTENER:

Lógica de negocio, validaciones de negocio

EJEMPLO:

`DbContext, RabbitMQ, Consumo de APIS`

TECNOLOGÍAS:

Entity Framework, Dapper, MongoDB drivers, KAFKA, SQS